<?php

MATERIAL DE ESTUDIO / DESARROLLO WEB

UNIDAD DIDÁCTICA 02

Tutorial 06
CRUD de productos

Repositorio, validación y persistencia con PDO

PHP · PDO · MySQL · Productos

APUNTES DE CLASE

Desarrollo web · PHP · Tutorial guiado

{ PHP }

UD 02 · DWES · PHP 8.1+

Tutorial 06 · CRUD de productos con PDO

Objetivo: Vamos a construir un pequeño catálogo de tienda con nombre, precio y stock. Podremos registrar productos, consultar el listado, cambiar precios y eliminar registros. Repetiremos el patrón PDO y repositorio, esta vez añadiendo validaciones numéricas.

Antes de comenzar

Utiliza PHP 8.1 o posterior y un servidor web local (por ejemplo, Apache de XAMPP). Crea una carpeta de proyecto dentro de `htdocs/UD2/` y **escribe los archivos con el contenido mostrado en cada paso**. El directorio `Codigo/` contiene esos mismos archivos listos para ejecutar.

Inicia **Apache y MySQL** desde XAMPP. Comprueba que PHP dispone de la extensión `pdo_mysql`. En los ejemplos se usa MySQL local con usuario `root` y contraseña vacía, una configuración habitual de laboratorio: si tu instalación utiliza otras credenciales, cámbialas en `config.php`. No utilices esas credenciales de prueba en un servidor público.

Crea `htdocs/UD2/Productos/` con `schema.sql`, `config.php`, `ProductoRepositorio.php` e `index.php`. La URL será `http://localhost/UD2/Productos/index.php`.

Paso 1. Crear la base de datos de productos

En phpMyAdmin, abre la pestaña SQL y ejecuta `schema.sql`. Se crea `ud2_tienda` y la tabla `producto`, con las columnas `id`, `nombre`, `precio` y `stock`.

El precio se almacena como `DECIMAL(10,2)` para representar importes con dos decimales. Las restricciones `CHECK` impiden precios negativos y cantidades de stock negativas; la validación del servidor debe existir igualmente para dar mensajes comprensibles y no depender únicamente de MySQL.

Archivo `Codigo/schema.sql`

```
CREATE DATABASE IF NOT EXISTS ud2_tienda
    CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;

USE ud2_tienda;

CREATE TABLE IF NOT EXISTS producto (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(120) NOT NULL,
    precio DECIMAL(10, 2) NOT NULL,
    stock INT NOT NULL DEFAULT 0,
    CONSTRAINT chk_precio CHECK (precio >= 0),
    CONSTRAINT chk_stock CHECK (stock >= 0)
);
```

La columna `stock` es un entero. No tiene sentido enviar `2.5` unidades cuando este modelo representa existencias indivisibles.

Paso 2. Conectar con MySQL mediante PDO

Crea `config.php`. La función `conectar()` devuelve la conexión a la base `ud2_tienda`. La configuración es la misma utilizada para usuarios, pero cambia el nombre de la base de datos.

`charset=utf8mb4` permite guardar caracteres como tildes o símbolos habituales en nombres de productos. Si tu MySQL tiene contraseña, modifica únicamente las credenciales de este archivo.

Archivo `Codigo/config.php`

```
<?php
declare(strict_types=1);

function conectar(): PDO
{
    return new PDO(
        'mysql:host=127.0.0.1;dbname=ud2_tienda;charset=utf8mb4',
        'root',
        '',
        [
            PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
            PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
            PDO::ATTR_EMULATE_PREPARES => false
        ]
    );
}
```

Todas las consultas posteriores usarán este PDO. Nunca se concatenan entradas del formulario para construir sentencias SQL.

Paso 3. Programar el repositorio de productos

Crea `ProductoRepositorio.php`. El constructor recibe PDO. `todos()` devuelve un array con los productos ordenados por ID descendente. `crear()` valida nombre, precio y stock antes de ejecutar un `INSERT` parametrizado.

`actualizarPrecio()` recibe el ID y el nuevo precio; no cambia el nombre ni el stock. `eliminar()` borra por ID. Los métodos de escritura se declaran `void` porque su tarea es modificar la base de datos; ninguno imprime mensajes en pantalla. El `SELECT` sí devuelve un array para mostrarlo en la vista.

Archivo `Codigo/ProductoRepositorio.php`

```
<?php
declare(strict_types=1);

class ProductoRepositorio
{
    public function __construct(private PDO $pdo) {}

    public function todos(): array
    {
        return $this->pdo
            ->query('SELECT id, nombre, precio, stock FROM producto ORDER BY id DESC')
            ->fetchAll();
    }

    public function crear(string $nombre, float $precio, int $stock): void
    {
        $nombre = trim($nombre);
        if ($nombre === '' || strlen($nombre) > 120
            || !is_finite($precio) || $precio < 0 || $stock < 0) {
            throw new InvalidArgumentException('Producto no válido.');
        }
        $sentencia = $this->pdo->prepare(
            'INSERT INTO producto(nombre, precio, stock) VALUES (?, ?, ?)'
        );
```

```
        $sentencia->execute([$nombre, $precio, $stock]);
    }

    public function actualizarPrecio(int $id, float $precio): void
    {
        if ($id < 1 || !is_finite($precio) || $precio < 0) {
            throw new InvalidArgumentException('Id o precio no válido.');
        }
        $sentencia = $this->pdo->prepare(
            'UPDATE producto SET precio = ? WHERE id = ?'
        );
        $sentencia->execute([$precio, $id]);
    }

    public function eliminar(int $id): void
    {
        if ($id < 1) {
            throw new InvalidArgumentException('Id no válido.');
        }
        $sentencia = $this->pdo->prepare('DELETE FROM producto WHERE id = ?');
        $sentencia->execute([$id]);
    }
}
```

La comprobación `!is_finite($precio)` impide valores numéricos especiales como infinito o NaN si la función fuese llamada desde otro código PHP; el `index.php` valida además el formato recibido por el formulario.

Paso 4. Procesar el formulario y representar los registros

Crea `index.php`. Primero importa la conexión y el repositorio y prepara la función `h()` para escapar texto HTML. Ante `POST`, lee el campo `accion` y distingue entre crear, actualizar y eliminar.

Para crear se usan `FILTER_VALIDATE_FLOAT` en el precio y `FILTER_VALIDATE_INT` en el stock; para actualizar se validan el ID y el precio; para eliminar se valida el ID. Después de una acción correcta se redirige a `index.php`. En una petición `GET` se consulta `todos()`.

El HTML muestra un formulario de alta y una lista de productos. Dentro de cada fila se generan dos formularios: uno con el nuevo precio y otro con el botón de eliminación. El precio y nombre recuperados de MySQL se representan de forma segura.

Archivo `Codigo/index.php`

```
<?php
declare(strict_types=1);
require_once __DIR__ . '/config.php';
require_once __DIR__ . '/ProductoRepositorio.php';

function h(string $valor): string
{
    return htmlspecialchars($valor, ENT_QUOTES, 'UTF-8');
}

$error = '';
$productos = [];
try {
    $repo = new ProductoRepositorio(conectar());

    if ($_SERVER['REQUEST_METHOD'] === 'POST') {
        $accion = (string) ($_POST['accion'] ?? '');
        $id = filter_var(
            $_POST['id'] ?? null,
            FILTER_VALIDATE_INT,
            ['options' => ['min_range' => 1]]
```

```
        );
        if ($accion === 'crear') {
            $nombre = trim((string) ($_POST['nombre'] ?? ''));
            $precio = filter_var($_POST['precio'] ?? null, FILTER_VALIDATE_FLOAT);
            $stock = filter_var(
                $_POST['stock'] ?? null,
                FILTER_VALIDATE_INT,
                ['options' => ['min_range' => 0]]
            );
            if ($precio === false || $stock === false) {
                throw new InvalidArgumentException('Precio o stock no válido.');
            }
            $repo->crear($nombre, $precio, $stock);
        } elseif ($accion === 'actualizar') {
            $precio = filter_var($_POST['precio'] ?? null, FILTER_VALIDATE_FLOAT);
            if ($id === false || $precio === false) {
                throw new InvalidArgumentException('Id o precio no válido.');
            }
            $repo->actualizarPrecio($id, $precio);
        } elseif ($accion === 'eliminar') {
            if ($id === false) {
                throw new InvalidArgumentException('Id no válido.');
            }
            $repo->eliminar($id);
        } else {
            throw new InvalidArgumentException('Acción no reconocida.');
        }
        header('Location: index.php');
        exit;
    }
    $productos = $repo->todos();
} catch (InvalidArgumentException $errorValidacion) {
    http_response_code(400);
    $error = $errorValidacion->getMessage();
} catch (PDOException $errorBD) {
    http_response_code(500);
    $error = 'Error de base de datos. Revisa la conexión y el servidor.';
}
?>
<!doctype html>
<html lang="es">
<head>
    <meta charset="utf-8">
    <title>Catálogo de productos</title>
</head>
<body>
    <h1>Catálogo de productos</h1>
    <?php if ($error !== ''): ?>
        <p><?= h($error) ?></p>
    <?php endif; ?>

    <h2>Nuevo producto</h2>
    <form action="index.php" method="post">
        <input type="hidden" name="accion" value="crear">
        <label>Nombre <input name="nombre" maxlength="120" required></label>
        <label>Precio <input name="precio" type="number" step="0.01" min="0" required></label>
        <label>Stock <input name="stock" type="number" step="1" min="0" required></label>
        <button type="submit">Crear</button>
    </form>

    <h2>Productos guardados</h2>
    <ul>
        <?php foreach ($productos as $producto): ?>
            <li>
                <?= h((string) $producto['nombre']) ?> —
                <?= h((string) $producto['precio']) ?> € —
                <?= (int) $producto['stock'] ?> unidades
                <form action="index.php" method="post" style="display:inline">
```

```
                    <input type="hidden" name="accion" value="actualizar">
                    <input type="hidden" name="id" value="<?= (int) $producto['id'] ?>">
                    <input type="number" name="precio" min="0" step="0.01"
                           value="<?= h((string) $producto['precio']) ?>" required>
                    <button type="submit">Actualizar precio</button>
                </form>
                <form action="index.php" method="post" style="display:inline">
                    <input type="hidden" name="accion" value="eliminar">
                    <input type="hidden" name="id" value="<?= (int) $producto['id'] ?>">
                    <button type="submit">Eliminar</button>
                </form>
            </li>
        <?php endforeach; ?>
    </ul>
</body>
</html>
```

El atributo `step="0.01"` orienta al navegador para introducir precios con dos decimales, pero no sustituye la validación del servidor. El precio `0` es válido y no debe confundirse con un dato ausente.

Resultado y comprobación

Abre la aplicación y registra `Cuaderno`, precio `2.50` y stock `15`: el catálogo mostrará esa fila. Modifica el precio a `3.20`: el nuevo importe aparecerá tras la redirección. Comprueba que el precio `0` también funciona. Intenta introducir un precio negativo o un stock fraccionario: el servidor o las restricciones de la tabla los rechazan. Elimina el cuaderno: deja de aparecer. Si MySQL no está iniciado, se mostrará el mensaje general de error de conexión, sin traza ni contraseña.

Lo importante: `DECIMAL` almacena importes en MySQL, el repositorio ejecuta sentencias preparadas y `index.php` valida los datos y produce el HTML.